Marevlo Research / AI Foundations • Module 02 – Quantization • Topic 5

FP8: Floating-Point at 8 Bits, Worked Out by Hand

Decoding an E4M3 byte by hand — and why a log-spaced grid beats INT8 for Gaussian weights

What this document does. It takes one FP8 E4M3 byte — the bits `0 1010 110` — and decodes it to a real number from first principles (= 14.0), then maps the entire E4M3 number line: bias, max finite (448), normals, subnormals, and the dynamic range (2^{18}). Finally it quantizes a realistic heavy-tailed weight tensor with both INT8 and FP8 and shows FP8 cuts RMSE by 56.9% — because its grid is fine where the data is, near zero. Every figure is reproduced in Appendix A (`m02_t05_fp8_engine.py`).

The one idea. INT8 spaces its 256 codes *uniformly* — every tick is the same distance apart. FP8 (like every floating-point format) spaces them *logarithmically* — dense near zero, sparse far out — because the exponent picks the scale and the mantissa fills in. LLM weights and activations are Gaussian-ish: most mass near zero, a few outliers. A log grid matches that shape; a uniform grid wastes resolution covering the outliers. That mismatch is why FP8 often beats INT8 with *zero* calibration.

Concepts covered, in order

the E4M3 and E5M2 layouts; the exponent bias; decoding a byte to a real value; normals vs. subnormals; max finite, min normal, min subnormal, and dynamic range; why FP8 spacing grows with magnitude; a head-to-head INT8-vs-FP8 error test on heavy-tailed weights; the FP8 KV-cache win.

Internal learning material. Numbers are reproducible from Appendix A. v1.0

Contents

1 The two FP8 formats

Hopper (H100) added native tensor-core support for two 8-bit floating-point formats. Both split a byte into sign / exponent / mantissa; they differ in how they trade range against precision.

Format	Layout (S·E·M)	Bias	Max finite	Use for
FP16	$1 \cdot 5 \cdot 10$	15	$\pm 65{,}504$	baseline
BF16	$1 \cdot 8 \cdot 7$	127	$\pm 3.4 \times 10^{38}$	training (FP32 range)
FP8 E4M3	$1 \cdot 4 \cdot 3$	7	± 448	**weights, activations**
FP8 E5M2	$1 \cdot 5 \cdot 2$	15	$\pm 57{,}344$	gradients (wider range)
INT8	signed integer	—	± 127	legacy uniform quant

Intuition

A floating-point number is scientific notation in binary: a sign, an *exponent* that says "which power-of-two neighbourhood are we in," and a *mantissa* that says "where in that neighbourhood." E4M3 spends 4 bits on the neighbourhood (so it can reach from 0.002 to 448) and 3 bits on the position within it. E5M2 trades one mantissa bit for an exponent bit: it reaches much farther (57,344) but resolves each neighbourhood more coarsely. Inference wants precision near the data, so E4M3 is the inference format.

2 Step 1 — Decode a byte by hand

The value of a *normal* E4M3 number (exponent field e from 1 to 14) is:

E4M3 normal value

$$v \;=\; (-1)^{s} \cdot 2^{\,e-\text{bias}} \cdot \left(1 + \tfrac{m}{8}\right), \qquad \text{bias} = 7, \;\; m \in \{0, \ldots, 7\}.$$

Decode 0 1010 110

Split the byte: sign $s = 0$, exponent field $e = 1010_2 = 10$, mantissa $m = 110_2 = 6$.

$$\begin{aligned} \text{unbiased exponent} &= e - \text{bias} = 10 - 7 = 3, \\ \text{mantissa factor} &= 1 + \tfrac{m}{8} = 1 + \tfrac{6}{8} = 1.75, \\ v &= (-1)^0 \cdot 2^3 \cdot 1.75 = 8 \times 1.75 = \mathbf{14.0}. \quad \checkmark \end{aligned}$$

3 Step 2 — The number line: normals, subnormals, range

3.1 Subnormals fill the gap at zero

When the exponent field is 0, the implicit leading 1 becomes a 0 and the exponent freezes at $2^{1-\text{bias}}$, giving *subnormal* values that pack tightly around zero:

E4M3 subnormal value

$$v = (-1)^s \cdot 2^{1-\text{bias}} \cdot \frac{m}{8} = (-1)^s \cdot 2^{-6} \cdot \frac{m}{8}.$$

3.2 The four landmark magnitudes

Landmark	Bits (S.EEEE.MMM)	Value
Max finite	0.1111.110	$1.75 \times 2^8 = 448$
Min normal	0.0001.000	$2^{-6} = 0.015625$
Min subnormal	0.0000.001	$\frac{1}{8} \times 2^{-6} = 0.001953$
NaN (reserved)	·.1111.111	— (no infinities in E4M3)

The dynamic range from the smallest subnormal to the largest finite is

$$\frac{448}{0.001953} = 229{,}376 = 2^{18},$$

all from 8 bits. By contrast INT8 has a fixed dynamic range of 127 ($\approx 2^7$). ✓

Watch out

E4M3 has **no infinities** — the all-ones exponent is reserved for NaN, and the max finite is 448, not a power of two. A value that overflows 448 does not saturate to "inf"; it becomes NaN and poisons the matmul. Production FP8 always applies a per-tensor scale so the tensor's max lands at (or just below) 448. Forgetting that scale is the classic FP8 footgun.

4 Step 3 — Why FP8 spacing grows with magnitude

Within one exponent band the 3 mantissa bits give 8 evenly spaced steps; the step size is $2^{e-\text{bias}}/8$. Each step *up* in exponent *doubles* that spacing:

Near value	exponent	E4M3 step
0.1250	2^{-3}	0.01562
0.2500	2^{-2}	0.03125
0.5000	2^{-1}	0.06250
1.0000	2^{0}	0.12500
2.0000	2^{1}	0.25000
4.0000	2^{2}	0.50000
8.0000	2^{3}	1.00000

Log grid vs. uniform grid

FP8's step is *tiny* near zero (0.0156 at magnitude 0.125) and *coarse* far out (1.0 at magnitude 8). INT8 has *one* step everywhere = $\max|W|/127$; a single far-out outlier forces that step to be coarse even for the tiny near-zero weights that are 99% of the tensor. FP8

spends precision where the data lives; INT8 spreads it uniformly, including over empty range it does not need.

5 Step 4 — INT8 vs. FP8 on a realistic weight tensor

Take 20,000 weights: a Gaussian core $\mathcal{N}(0, 0.05)$ plus 0.2% outliers near ± 2 — the real transformer shape (dense near zero, heavy tail). Quantize with INT8 (uniform) and FP8 E4M3 (log), each scaled so the tensor max fits, and compare RMSE.

Scheme	RMSE	Why
INT8 (uniform)	0.005667	step $= 2.484/127 = 0.0196$ *everywhere*; outliers coarsen the bulk
FP8 E4M3 (log)	0.002441	step $\ll 0.0196$ near zero, where the mass is

The verdict

The outlier at ± 2.48 sets INT8's uniform step to 0.0196, so a typical weight of 0.05 is resolved to about two ticks. FP8, scaled the same way, still has steps of order 10^{-3} near zero, so it resolves the bulk an order of magnitude finer. Result:

$$\text{RMSE}: \quad 0.005667 \text{ (INT8)} \longrightarrow 0.002441 \text{ (FP8)}, \qquad \textbf{a } 56.9\% \textbf{ reduction.} \quad \checkmark$$

And FP8 needs *no calibration* — there are no scale factors to fit beyond the single per-tensor max, because the format itself is shaped like the data.

Why FP8 sometimes beats INT8 by 1–2 perplexity points

On Gaussian-ish LLM tensors, FP8 PTQ often beats INT8 SmoothQuant by 1–2 perplexity, with zero calibration data, because a floating-point grid is *intrinsically* matched to a heavy-tailed distribution. A single large value burns one exponent slot, not a huge fraction of the representable range — so FP8 handles outliers gracefully where INT8 needs SmoothQuant (Topic 4) to survive them.

6 Step 5 — The underappreciated win: FP8 KV cache

Weights are not the only big tensor. In long-context serving the KV cache often exceeds the weights. A single Nexus request (6,200 input tokens, Llama-3-70B) holds roughly

$$8192 \times 80 \,\text{layers} \times 2\,(K, V) \times 8\,(\text{GQA}) \times 2\,\text{B} \times 6200 \;\approx\; 14\,\text{GB per request in FP16.}$$

Quantizing that KV cache to FP8 halves it to $\sim$7 GB with no measurable accuracy loss, roughly *doubling* the concurrent requests per GPU.

Watch out

FP8 vs. INT4 is a hardware-dependent choice, not a dogma. On H100: FP16 TPOT 22 ms, FP8 7 ms (140 $\rightarrow$ 70 GB), INT4-AWQ 6 ms ($\rightarrow$ 38 GB). INT4 is smaller and slightly faster; FP8 keeps slightly better accuracy and quantizes activations and KV cache cleanly. On B200 with native FP8 kernels, FP8 closes most of the speed gap. Pick by hardware and accuracy budget.

7 The argument, at a glance

#	Step	Result
1	Decode	$0\,1010\,110 = 2^3 \cdot 1.75 = 14.0$
2	Range	max 448, min subnormal 0.00195, range 2^{18}, no inf
3	Spacing	step $= 2^{e-7}/8$, doubles per band; fine near 0
4	vs. INT8	heavy-tail RMSE 0.00567 $\rightarrow$ 0.00244 (-56.9%)
5	KV cache	FP8 halves $\sim$14 GB $\rightarrow$ 7 GB, 2$\times$ concurrency

7.1 Equation cheat-sheet

Normal value	$v = (-1)^s\, 2^{\,e-\text{bias}}\,(1 + m/2^M)$
Subnormal value	$v = (-1)^s\, 2^{\,1-\text{bias}}\,(m/2^M)$
E4M3 bias	7 (E5M2 bias 15)
Step in band e	$2^{\,e-\text{bias}}/2^M$
Max finite (E4M3)	$1.75 \times 2^8 = 448$
Per-tensor FP8 scale	$s = \max\lvert M\rvert/448$

Appendix A — Reference implementation

Every number above is produced by `m02_t05_fp8_engine.py`. Run it to reproduce the byte decode, the dynamic-range landmarks, the doubling spacing table, and the INT8-vs-FP8 error test; change the byte fields or the weight distribution to explore.

```
"""
Reference implementation for: Module 02 - Topic 5
"FP8 - floating point at 8 bits (E4M3 / E5M2)."

We decode a specific E4M3 byte to a real number from first principles, map out
the E4M3 dynamic range and subnormals, then compare INT8 (uniform) vs FP8
(log-spaced) quantization error on a Gaussian weight distribution -- the reason
FP8 beats INT8 for LLM weights.

Every number printed here is transcribed (rounded) into the worked-by-hand PDF.
"""

import numpy as np

np.set_printoptions(precision=4, suppress=True)

# ---------------------------------------------------------------------------
# E4M3 format: 1 sign bit, 4 exponent bits, 3 mantissa bits. bias = 7.
#   normal    (1 <= e <= 14):  (-1)^s * 2^(e-7) * (1 + m/8)
#   subnormal (e == 0):        (-1)^s * 2^(1-7) * (0 + m/8) = 2^-6 * (m/8)
#   e == 15 is reserved (NaN); E4M3 has no infinities (max finite = 448).
# ---------------------------------------------------------------------------
BIAS = 7
M_BITS = 3
M_LEVELS = 2 ** M_BITS          # 8


def decode_e4m3(sign, exp, mant):
    """Decode the three fields of an E4M3 value to a real number."""
    if exp == 0:                                    # subnormal
        val = (mant / M_LEVELS) * 2.0 ** (1 - BIAS)
```

```
    else:                                          # normal
        val = (1 + mant / M_LEVELS) * 2.0 ** (exp - BIAS)
    return (-1) ** sign * val


# ------------------------------------------------------------------------
# 1. Decode one specific byte:  0 1010 110  (sign=0, exp=1010b=10, mant=110b=6)
# ------------------------------------------------------------------------
sign, exp, mant = 0, 0b1010, 0b110
val = decode_e4m3(sign, exp, mant)
print("== 1. Decode the E4M3 byte  0 1010 110 ==")
print(f"sign={sign}  exp=1010b={exp}  mant=110b={mant}")
print(f"unbiased exponent e - bias = {exp} - {BIAS} = {exp - BIAS}")
print(f"mantissa value 1 + m/8 = 1 + {mant}/8 = {1 + mant/M_LEVELS}")
print(f"value = (-1)^{sign} * 2^{exp-BIAS} * {1+mant/M_LEVELS} = {val}")

# ------------------------------------------------------------------------
# 2. Dynamic range of E4M3.
# ------------------------------------------------------------------------
max_normal = decode_e4m3(0, 15 - 1, M_LEVELS - 1)   # e=14, m=7 (e=15 is NaN)
# careful: e=15,m=7 would be NaN; max finite uses e=15 except all-ones=NaN.
# Standard OCP E4M3: max finite = 448 (S.1111.110). Recompute that exactly:
max_finite = (1 + 6 / 8) * 2 ** (15 - BIAS)         # e=15, m=6 -> 1.75 * 256
min_normal = decode_e4m3(0, 1, 0)                   # e=1, m=0
min_subnormal = decode_e4m3(0, 0, 1)               # e=0, m=1
print("\n== 2. E4M3 dynamic range ==")
print(f"max finite (S.1111.110)   = 1.75 * 2^{15-BIAS} = {max_finite}")
print(f"min normal (S.0001.000)   = 2^{1-BIAS}          = {min_normal}")
print(f"min subnormal (S.0000.001)= (1/8) * 2^{1-BIAS}  = {min_subnormal}")
print(f"dynamic range max/min_sub = {max_finite/min_subnormal:.0f}x "
      f"= 2^{np.log2(max_finite/min_subnormal):.0f}")

# ------------------------------------------------------------------------
# 3. Representable values near zero: show the FIRST few positive E4M3 grid
#    points and their spacing (log-spaced!) vs INT8 (uniform).
# ------------------------------------------------------------------------
print("\n== 3. E4M3 spacing GROWS with magnitude (log-spaced) ==")
# Show the step size right ABOVE a few powers of two. Within one exponent
# band the step is constant (= 2^(e-bias) / 8); it DOUBLES each band.
for e in range(4, 11):
    lo = decode_e4m3(0, e, 0)          # 2^(e-bias)
    step = decode_e4m3(0, e, 1) - lo   # one mantissa tick at this magnitude
    print(f"  near value {lo:8.4f} (2^{e-BIAS:+d}):  E4M3 step = {step:.5f}")
print("  -> each band up DOUBLES the step: fine near 0, coarse far out.")

# INT8 for comparison: uniform grid over the tensor's range.
print("\nINT8: ONE uniform step over the whole range, = absmax/127 EVERYWHERE")
print("  so a far-out outlier forces a COARSE step even for tiny near-zero weights.")

# ------------------------------------------------------------------------
# 4. The payoff: quantize a HEAVY-TAILED weight tensor with INT8 vs FP8 E4M3.
#    Bulk ~ N(0, 0.05) (most weights), plus a handful of outliers ~ +-2.0.
#    This is the realistic transformer case: Gaussian core + a few big values.
#    INT8's uniform grid is STRETCHED by the outliers, coarsening the bulk;
#    FP8's log grid stays fine near zero where 99% of the mass lives.
# ------------------------------------------------------------------------
rng = np.random.default_rng(0)
Wt = rng.normal(0, 0.05, size=20000).astype(np.float64)
n_out = 40                                      # 0.2% outliers
idx_out = rng.choice(Wt.size, n_out, replace=False)
Wt[idx_out] = rng.choice([-1.0, 1.0], n_out) * rng.uniform(1.5, 2.5, n_out)
absmax = np.abs(Wt).max()

```

```
95
96  def quant_int8(w, absmax):
97      s = absmax / 127.0
98      return np.clip(np.round(w / s), -127, 127) * s
99
100
101 def quant_e4m3(w, absmax):
102     # Scale so the tensor max lands at E4M3 max finite (448), quantize, unscale.
103     s = absmax / max_finite
104     ws = w / s
105     # round each value to nearest E4M3 grid point
106     allgrid = []
107     for e in range(0, 16):
108         for m in range(M_LEVELS):
109             if e == 15 and m == 7:
110                 continue                # NaN slot
111             allgrid.append(decode_e4m3(0, e, m))
112     allgrid = np.array(sorted(set(allgrid)))
113     full = np.concatenate([-allgrid[::-1], allgrid])
114     idx = np.abs(ws[:, None] - full[None, :]).argmin(axis=1)
115     return full[idx] * s
116
117
118 Wi = quant_int8(Wt, absmax)
119 Wf = quant_e4m3(Wt, absmax)
120 rmse_int8 = np.sqrt(np.mean((Wt - Wi) ** 2))
121 rmse_fp8 = np.sqrt(np.mean((Wt - Wf) ** 2))
122 print("\n== 4. Heavy-tailed weights (N(0,0.05) + ~0.2% outliers): INT8 vs FP8 RMSE ==")
123 print(f"tensor absmax = {absmax:.4f}")
124 print(f"INT8 RMSE = {rmse_int8:.6f}")
125 print(f"FP8  RMSE = {rmse_fp8:.6f}")
126 print(f"FP8 lowers RMSE by {(1-rmse_fp8/rmse_int8)*100:.1f}% on this Gaussian tensor")
127 print("  reason: most weights sit near 0, where FP8's log grid is much finer than INT8's uniform grid.")
128
129 # ---------------------------------------------------------------------
130 # 5. E5M2 contrast (wider range, coarser mantissa).
131 # ---------------------------------------------------------------------
132 print("\n== 5. E4M3 vs E5M2 ==")
133 print("  E4M3: 1-4-3, bias 7,  max +-448,    finer mantissa -> weights/activations")
134 print("  E5M2: 1-5-2, bias 15, max +-57344,  wider range    -> gradients (training)")
```

— END OF DOCUMENT —